

PriceIQ

AI-Powered Electronics Price Comparison for Indian Consumers

System Design Document

Document Version 1.0

Project Type:	BTech Final Year Project
Department:	Computer Science and Engineering
Academic Year:	2025–2026
Prepared By:	Adrij Bhadra
Date:	April 2026
Document Status:	Final

Covers: Client-Server Architecture, ER Diagram, Data Flow Diagrams,
Component Design, API Design, and Database Schema.

Contents

1	Introduction	3
1.1	Purpose	3
1.2	Scope	3
1.3	Design Goals	3
2	Client-Server Architecture	5
2.1	Overview	5
2.2	Architecture Diagram	5
2.3	Technology Stack	5
2.4	Request-Response Flow for Live Search	6
3	Entity-Relationship Diagram	8
3.1	Database Schema Overview	8
3.2	ER Diagram	8
3.3	Entity Descriptions	8
3.3.1	Product	8
3.3.2	ProductListing	8
3.3.3	PriceHistory	8
3.3.4	AIAnalysis	9
3.3.5	SearchLog	9
4	Data Flow Diagrams	10
4.1	DFD Level 0 — Context Diagram	10
4.2	DFD Level 1 — Main Processes	11
4.3	DFD Level 2 — P2 Live Scraper (Internal Flow)	12
4.4	DFD Level 2 — P3 Product Matcher (Scoring Decision Tree)	13
5	Component Design	14
5.1	Scraper Subsystem	14
5.2	UI Component Hierarchy	15

6	API Design	16
6.1	Score Engine Public Interface	17

Chapter 1

Introduction

1.1 Purpose

This System Design Document (SDD) describes the architectural decisions, component structures, data models, and data-flow characteristics of **PriceIQ**. It complements the Software Requirements Specification and provides the technical blueprint used during implementation.

1.2 Scope

The document covers the following design artefacts:

- Client-Server architecture and technology stack
- Entity-Relationship (ER) diagram for the database schema
- Data Flow Diagrams (DFD) at Context, Level-1, and Level-2
- Component-level design of the scraper subsystem
- REST API endpoint design
- Database schema with constraints

1.3 Design Goals

1. **Accuracy** — The product matching engine must reject accessories, wrong storage variants, and wrong model tiers before any data is stored.
2. **Modularity** — Each scraper is an independent module; the scoring engine is shared via a pure-function interface.
3. **Graceful Degradation** — A failed scraper must not prevent results from the other scrapers from being returned.
4. **Simplicity** — Next.js API routes co-locate server logic with the UI framework,

minimising operational overhead for a single-developer project.

Chapter 2

Client-Server Architecture

2.1 Overview

PriceIQ follows a **three-tier client-server architecture**:

1. **Presentation Tier** — React 19 components rendered server-side (SSR) by Next.js 16 and delivered to the user's browser.
2. **Application Tier** — Next.js API routes act as the backend, orchestrating scraping, database operations, and AI inference.
3. **Data Tier** — PostgreSQL 16 database managed by Prisma 7 ORM, running in a Docker container.

External services (Groq API, Amazon India, Flipkart, Vijay Sales) are consumed by the Application Tier only; the browser never communicates with them directly.

2.2 Architecture Diagram

2.3 Technology Stack

Layer	Technology	Rationale
Frontend Framework	Next.js 16.2.4	App Router enables per-route server components and streaming SSR; eliminates a separate backend server.
UI Library	React 19.2.4	Server and client components; concurrent rendering.
Styling	Tailwind CSS 4	Utility-first; zero runtime CSS; dark-mode via <code>dark:</code> variants.

Layer	Technology	Rationale
UI Components	Radix UI + shadcn/ui	Accessible, unstyled primitives; customised with Tailwind.
Charts	Recharts 3.8.1	Declarative React charting; price history line chart and dashboard pie/bar charts.
ORM	Prisma 7.7.0	Type-safe schema-first ORM; migrations via <code>db push</code> .
Database	PostgreSQL 16	ACID-compliant relational DB; full-text search via <code>ilike</code> .
Browser Automation	Playwright 1.59.1	Chromium headless browser for Flipkart DOM scraping with stealth headers.
LLM Inference	Groq SDK 1.1.2	Hardware-accelerated inference; LLaMA-3.3-70b at very low latency.
Containerisation	Docker + Compose	Reproducible environment; Postgres volume for persistent data.

2.4 Request-Response Flow for Live Search

The complete flow for a live search request is as follows:

1. User types a query in the browser and submits the search form.
2. Browser sends GET to `/search?q={query}`.
3. Next.js server component queries the database. If results are found, they are rendered and returned (**cache hit path**).
4. If no results exist, the `LiveSearchTrigger` client component is rendered.
5. The client component fires POST to `/api/search/live`.
6. The API route calls `liveSearch(query)`.
7. Vijay Sales (GraphQL) and Amazon (HTTP) scrapers execute in parallel via `Promise.allSettled`; Flipkart (Playwright) runs after.
8. Each scraper passes candidate titles through `scoreCandidate()` and returns the best-accepted `NormalizedProduct`.
9. Accepted products are upserted into the database; price history is recorded.
10. `analyzeProduct()` is called to produce AI analysis via Groq.
11. The API returns the new `productId`; client redirects to `/product/{id}`.
12. The product page renders server-side with current listings and AI analysis.

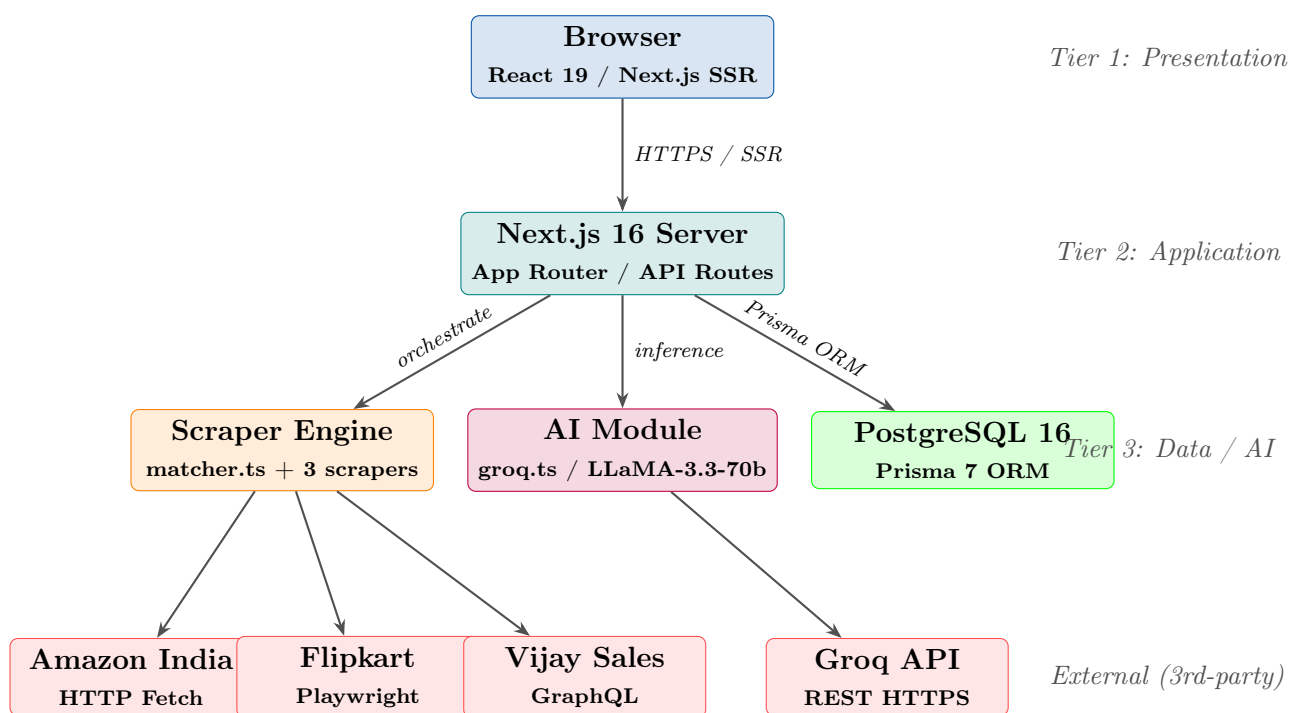


Figure 2.1: PriceIQ Three-Tier Client-Server Architecture

Chapter 3

Entity-Relationship Diagram

3.1 Database Schema Overview

The PriceIQ database consists of five entities. All IDs are auto-generated integers (Prisma default serial). All monetary values are stored as `Float` in INR.

3.2 ER Diagram

3.3 Entity Descriptions

3.3.1 Product

The central entity of the system. Each row represents one canonical electronics product identified by a unique, URL-safe slug (e.g., `iphone-15-128gb`). One product may have listings from one, two, or all three retail platforms.

3.3.2 ProductListing

Each row represents the real-time state of a product on a specific retail platform. The composite unique key (`productId`, `source`) ensures that a re-scrape updates the existing row rather than inserting a duplicate. The `extraData` JSON column stores platform-specific attributes such as Vijay Sales MRP.

3.3.3 PriceHistory

An append-only table that records every price observation. A new row is created on every scrape event (both live and batch). This table is the data source for the price-trend line chart on the product page.

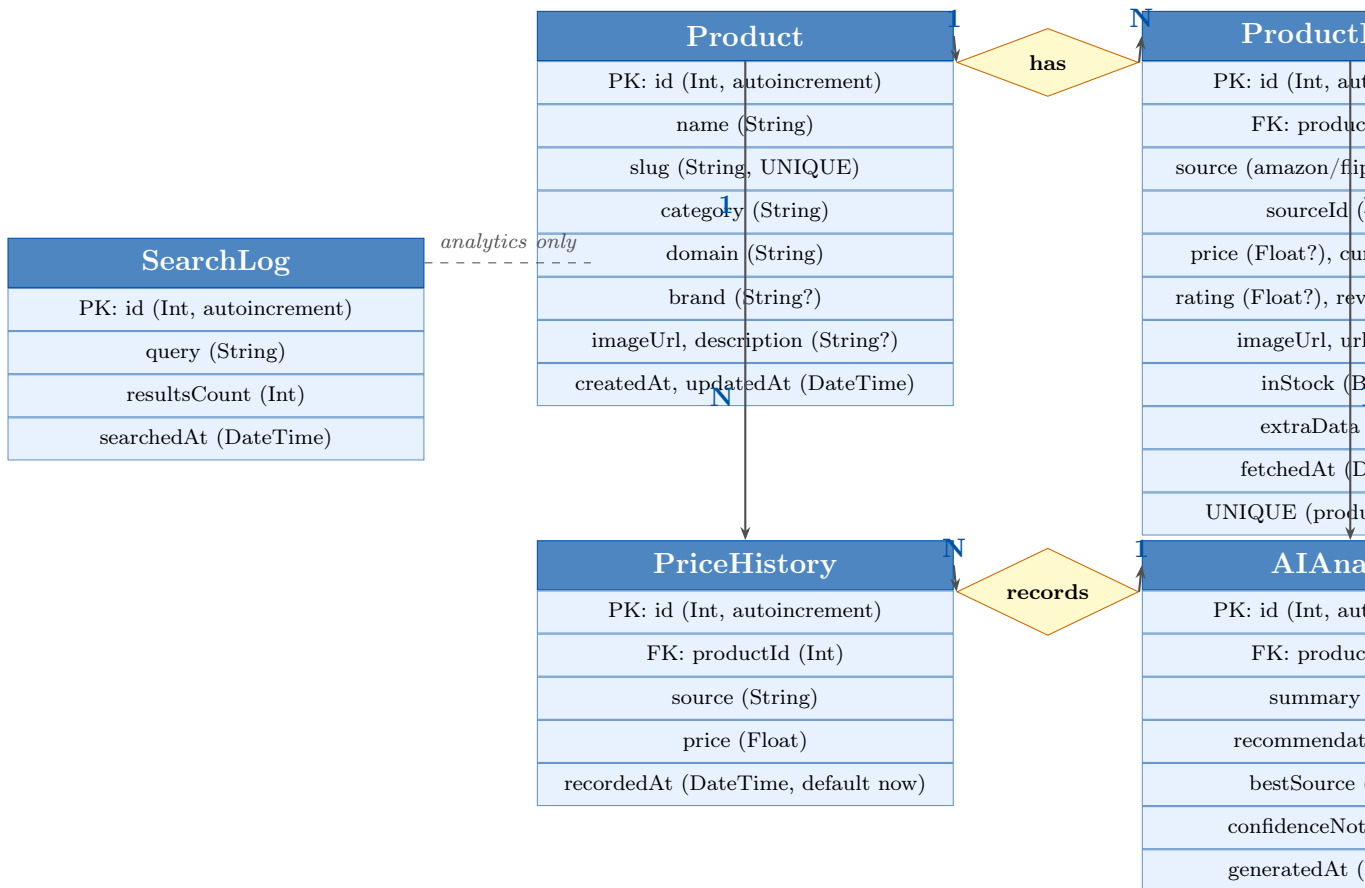


Figure 3.1: PriceIQ Entity-Relationship Diagram

3.3.4 AIAnalysis

Stores the structured LLM output. Multiple rows per product are allowed; the application always reads the most recent row (sorted by **generatedAt** descending). The 24-hour cache TTL is enforced at the application layer, not the database.

3.3.5 SearchLog

A simple analytics log. Each user search appends one row. No personally identifiable information is stored (no user IDs or IP addresses).

Chapter 4

Data Flow Diagrams

4.1 DFD Level 0 — Context Diagram

The context diagram shows PriceIQ as a single process and its data exchanges with all external entities.

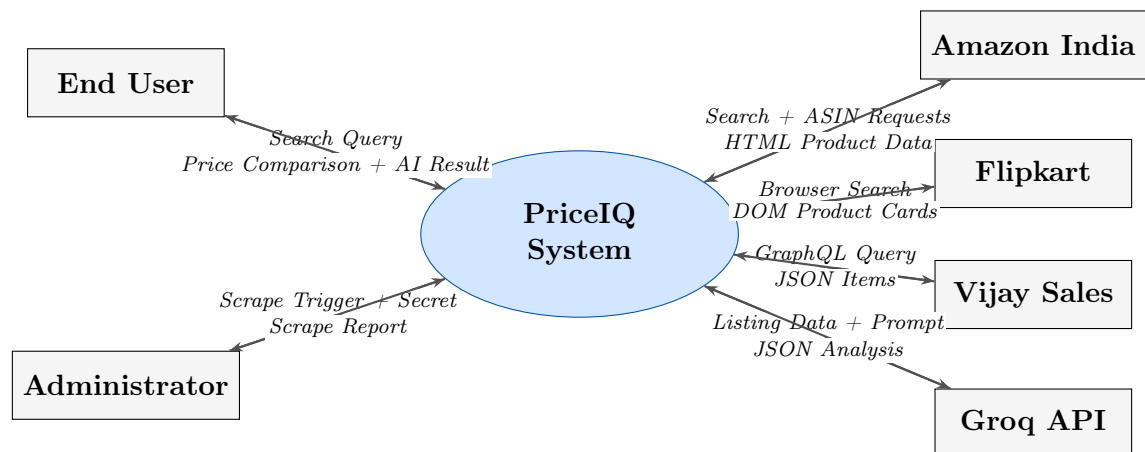


Figure 4.1: DFD Level 0 — PriceIQ Context Diagram

4.2 DFD Level 1 — Main Processes

The Level-1 diagram decomposes PriceIQ into five primary processes and two data stores.

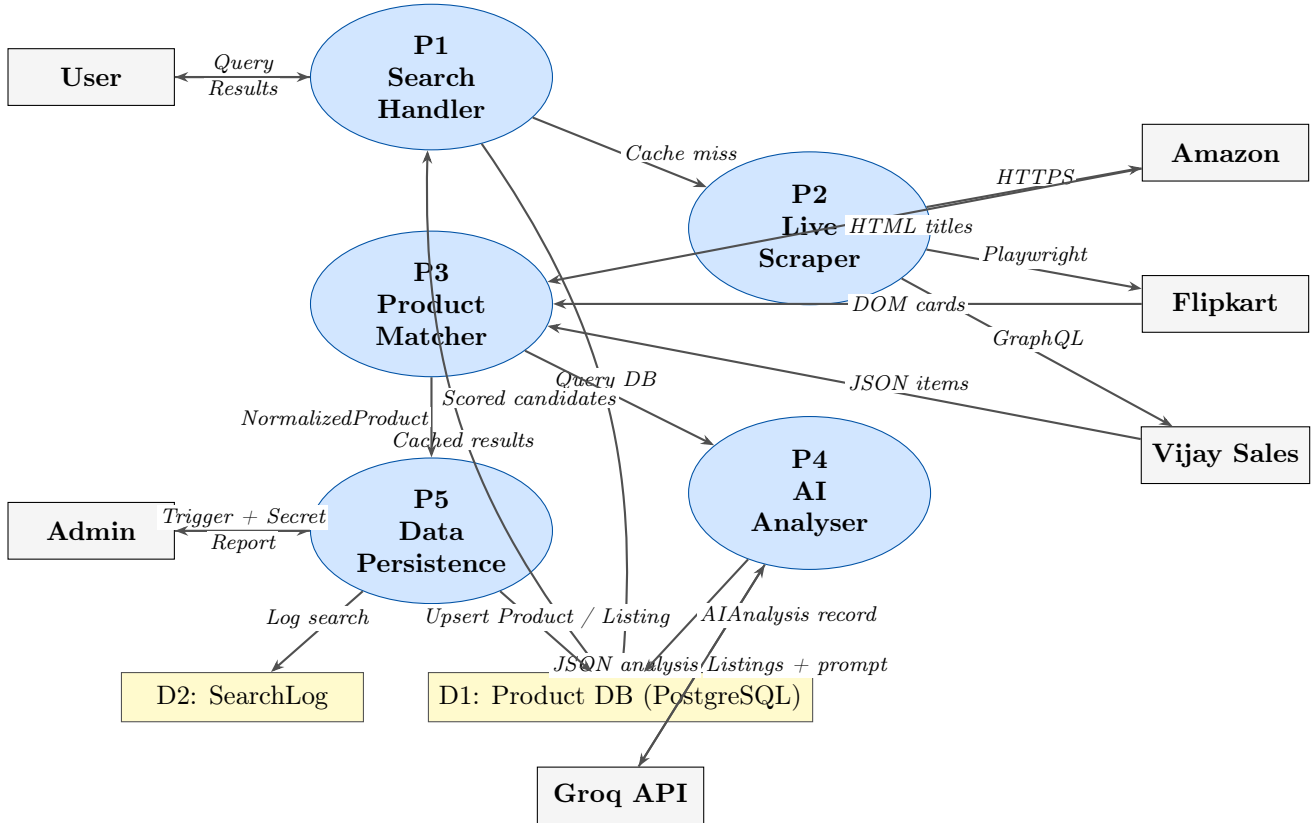


Figure 4.2: DFD Level 1 — Main Processes

4.3 DFD Level 2 — P2 Live Scraper (Internal Flow)

This diagram expands P2 (Live Scraper) to show the internal parallelism and sequential flow between the three platform scrapers.

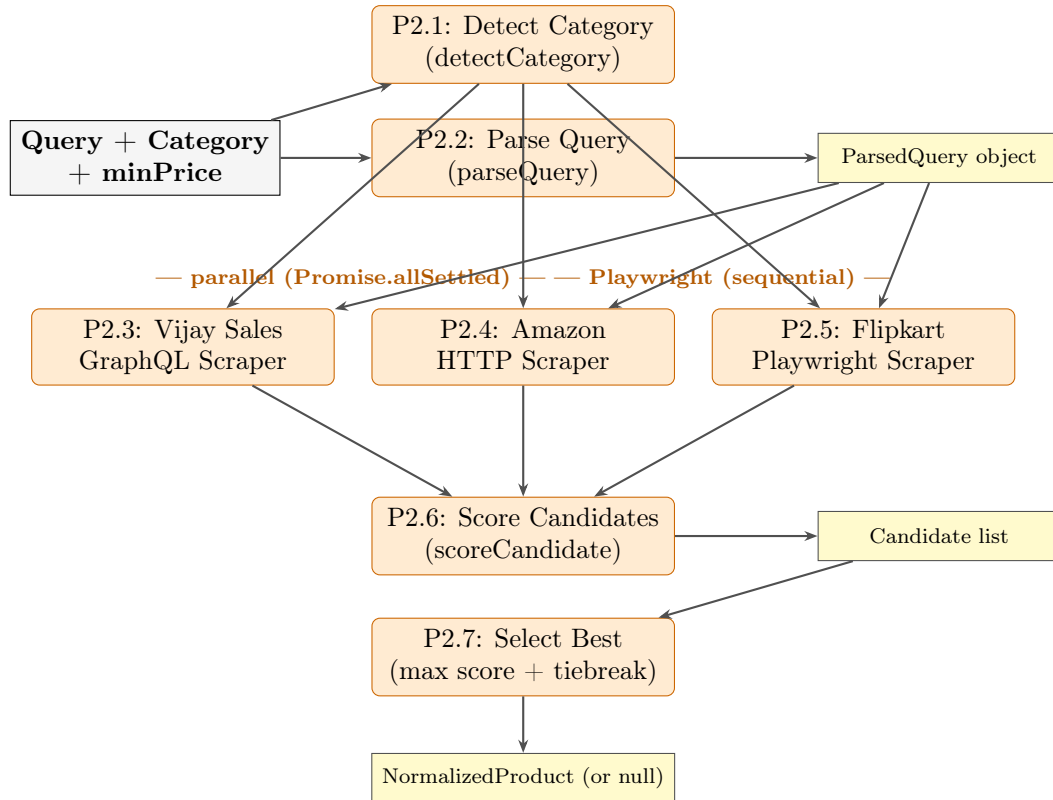


Figure 4.3: DFD Level 2 — P2 Live Scraper Internal Flow

4.4 DFD Level 2 — P3 Product Matcher (Scoring Decision Tree)

This diagram expands P3 (Product Matcher) to show the scoring decision tree implemented in `matcher.ts`.

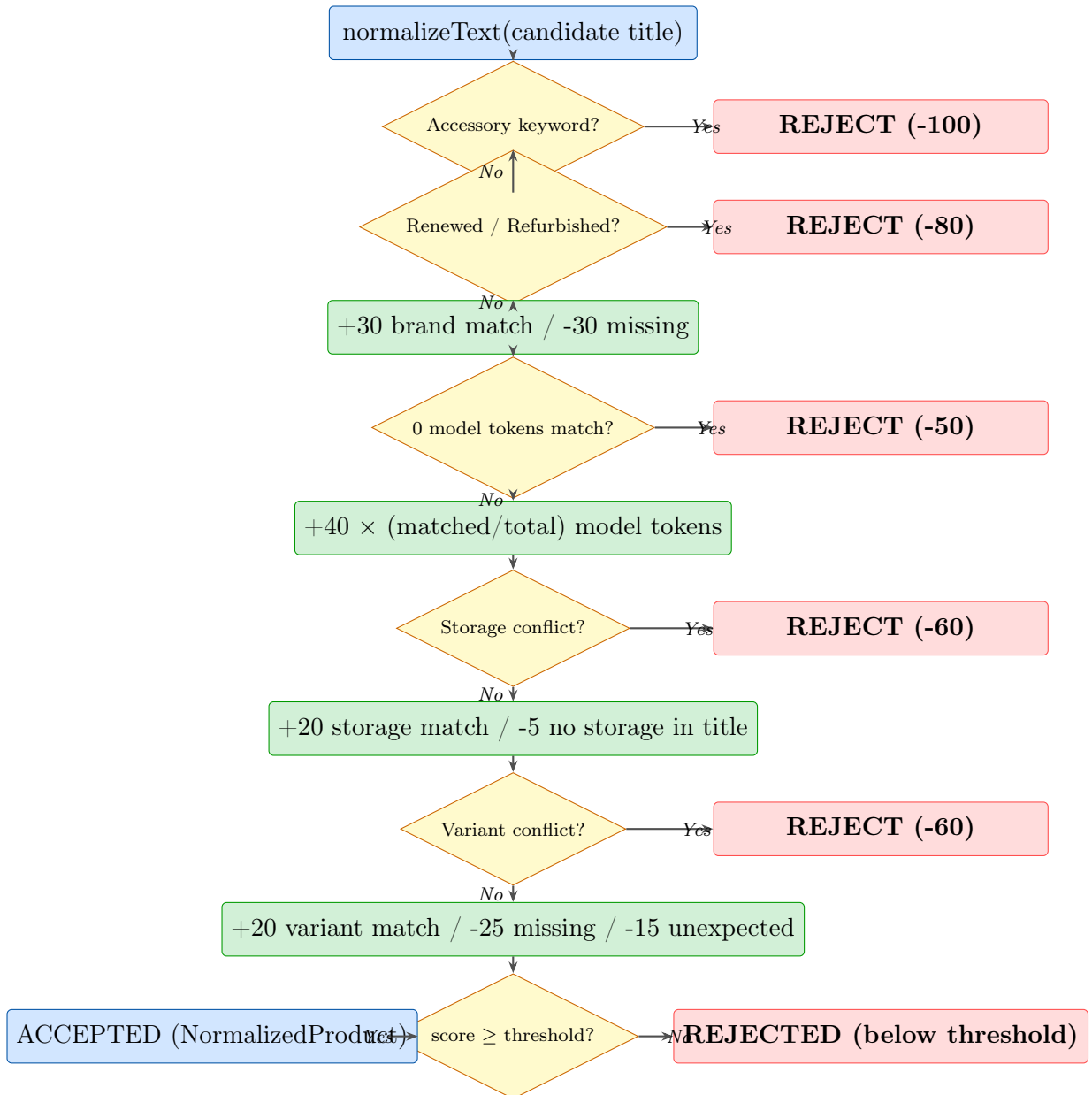


Figure 4.4: DFD Level 2 — P3 Product Matcher Scoring Decision Tree

Chapter 5

Component Design

5.1 Scraper Subsystem

The scraper subsystem is located at `src/lib/scrapers/` and is composed of the following modules:

Module	Responsibility
<code>matcher.ts</code>	Pure-function scoring engine. Exports <code>normalizeText</code> , <code>parseQuery</code> , and <code>scoreCandidate</code> . No side effects, no I/O. Can be unit-tested in isolation.
<code>products.ts</code>	Static catalogue of 4 products with per-source query strings and minimum price thresholds. Read-only data; no logic.
<code>vijaysales.ts</code>	Batch scraper: iterates <code>PRODUCTS</code> , calls the VS GraphQL endpoint, scores candidates, returns <code>NormalizedProduct []</code> .
<code>amazon.ts</code>	Batch scraper: extracts ASINs from Amazon search HTML, fetches each product page, scores, returns best.
<code>flipkart.ts</code>	Batch scraper: one Playwright browser shared across all products; evaluates DOM to extract candidate cards; scoring in Node.js.
<code>live.ts</code>	On-demand scraper: <code>liveSearch(query)</code> orchestrates all three live scraper functions and returns aggregated results.
<code>index.ts</code>	Exports <code>runAllScrapers()</code> for the batch scrape API route.

Module	Responsibility
<code>types.ts</code>	TypeScript interfaces (<code>NormalizedProduct</code> , <code>SourceName</code>). No runtime code.

5.2 UI Component Hierarchy

Component	Responsibility
<code>layout.tsx</code>	Root layout: <code>ThemeProvider</code> , <code>Navbar</code> , <code>Sonner</code> toast container.
<code>page.tsx</code> (home)	Marketing landing: <code>HeroSection</code> , <code>StatsBar</code> , <code>FeaturesGrid</code> , <code>CategoryGrid</code> .
<code>search/page.tsx</code>	SSR results page: <code>SearchBar</code> , <code>SearchFilters</code> , <code>ProductCard</code> grid, <code>LiveSearchTrigger</code> fallback.
<code>LiveSearchTrigger.tsx</code>	Client component: POST to <code>/api/search/live</code> , animated per-platform status, redirect on success.
<code>product/[id]/page.tsx</code>	SSR product detail: tabs for <code>PriceComparisonTable</code> and <code>AIRecommendation</code> .
<code>PriceComparisonTable.tsx</code>	Renders sorted listing cards with best-deal highlight and price-diff percentage.
<code>AIRecommendation.tsx</code>	Client component: lazy-fetches AI analysis; shows skeleton then result.
<code>PriceHistoryChart.tsx</code>	Recharts <code>LineChart</code> fetching <code>/api/price-history/[id]</code> .
<code>dashboard/page.tsx</code>	<code>KPICard</code> , <code>SourcePieChart</code> , <code>CategoryBarChart</code> , <code>RecentSearches</code> table.

Chapter 6

API Design

All API routes are implemented as Next.js Route Handlers under `src/app/api/`. JSON is the only data format. Error responses use standard HTTP status codes with a JSON body `{"error": "..."}.`

Method	Path	Auth?	Description
POST	/api/search/live	No	Run live scrape for a query. Body: <code>{"query": "..."}.</code> Returns <code>{productId, aiAnalysis}</code> .
GET	/api/search	No	Full-text product DB search. Params: <code>q</code> , <code>category</code> , <code>source</code> , <code>sort</code> . Returns product array.
GET	/api/search/suggestions	No	Returns up to 5 product name suggestions matching <code>q</code> .
GET	/api/products	No	Paginated product list. Params: <code>page</code> , <code>limit</code> .
GET	/api/products/[id]	No	Single product with all listings.
GET	/api/product/[id]	No	Product detail including listings and latest AI analysis.
POST	/api/product/[id]/refresh	No	Trigger re-scrape for a specific product by ID.

Method	Path	Auth?	Description
POST	/api/ai/analyze	No	Generate or refresh AI analysis. Body: {productId}.
GET	/api/price-history/[id]	No	Full price history array for a product.
GET	/api/dashboard	No	Aggregated KPI statistics for the dashboard page.
POST	/api/scrape	Yes (x-scraper-secret)	Trigger full batch scrape. Returns {upserted: N}.

6.1 Score Engine Public Interface

Function Signature	Description
parseQuery(raw, category, minPrice) -> ParsedQuery	Extracts structured intent from a raw search string: brand, model tokens, storageToken, variantToken. Passed to all scraper scoring calls.
scoreCandidate(title, parsedQ) -> MatchResult	Returns {score, accepted, reasons}. accepted is the gate for DB storage.
normalizeText(text) -> string	Lowercases, joins model codes (e.g. WH-1000XM5 to wh1000xm5), removes non-alphanumeric characters.

Revision History

Version	Date	Author	Changes
1.0	April 2026	Adrij Bhadra	Initial release covering architecture, ER diagram, three-level DFDs, component design, API design, and database schema.